

HRC

Azure Cloud Infrastructure

Development Environment

Version 1.0 | West US 2 | June 2026

CONFIDENTIAL - FOR INTERNAL USE ONLY

Infrastructure modernization using Terraform and Azure Container Apps (dev environment: HRC-dev)

Author: Cybersenz / HRC Engineering

Date: 3rd June 2026

Version: 1.0 - Draft for review

1. Executive Summary

Overview

HRC is a healthcare outbound platform designed for automated insurance verification. The system handles voice-based calls with interactive voice response (IVR) navigation, chat-based verification with major payers such as Aetna, structured benefits extraction, and automated PDF report generation.

Current State

Today, HRC operates as a production system running on a single Azure Virtual Machine within the resource group HRC. The platform uses Docker Compose to orchestrate approximately 13 containers across three application repositories. The system leverages shared Azure services including Azure OpenAI, Azure Storage, existing Key Vault, and external integrations with Twilio, ElevenLabs, and Azure Communication Services.

Target State

We are building a cloud-native development environment in a new resource group HRC-dev, provisioned entirely through Terraform from the HRC-infra repository. The new architecture uses Azure Container Apps (ACA) as the container orchestration platform, replaces self-hosted databases with Azure Database for PostgreSQL Flexible Server and Azure Cache for Redis, introduces a dedicated Azure Container Registry (ACR), provisions a new Key Vault for dev secrets, and implements comprehensive observability through Log Analytics and Application Insights.

Guiding Principle

The production resource group HRC remains completely isolated and unmodified during this migration phase. The dev environment operates with separate resource groups, distinct secrets, and independent data planes. This parallel-run approach ensures zero impact to production operations.

Objective

This migration represents infrastructure modernization and operational automation - not a redesign of application business logic. Voice agents (handling WebSockets, Twilio media streams, ElevenLabs integration) and chat agents will continue to function as they do today, but on superior Azure managed services with better scalability, reliability, and observability.

2. Current Architecture (As-Is)

2.1 Deployment Model

The current production deployment runs on a single Azure Virtual Machine (conceptually named hrc-server) within resource group HRC in the westus2 region. All application components are containerized and orchestrated via Docker Compose directly on the VM.

An Nginx reverse proxy container on the VM provides:

- TLS termination
- Path-based routing (/ , /api/* , /provider/* , /womens-health/*)
- Rate limiting
- WebSocket upgrade support for Twilio media streams

The platform is accessible via a public hostname (example: hrc-outbound.cybersenz.com).

2.2 Application Repositories

The platform is built from three primary repositories:

Repository	Purpose	Main Runtime Components
hrc-outbound-webapp	Dashboard + API orchestration	Next.js frontend (port 3000), FastAPI backend (8000), PDF worker, scheduler/sweeper, Nginx
hrc-voice-agents	Outbound insurance verification calls	Provider agent, Women's Health agent, dedicated Redis
hrc-chat-agent	Chat-based payer verification (e.g., Aetna)	Playwright + Azure OpenAI chat agent

2.3 Container Stack

Approximately 13 containers run on the VM across two Docker Compose projects:

Outbound webapp stack (single compose project):

- app - Next.js production build
- fastapi - FastAPI API server (Gunicorn/Uvicorn workers)
- pdf-worker - Background PDF generation jobs (Redis queue-based)
- postgres - PostgreSQL 15 in Docker (with SSL initialization)
- redis - Redis 7.4 in Docker (TLS enabled)
- nginx - Reverse proxy (ports 80/443)

Voice agents stack (separate compose project):

- provider-agent - Port 8001 mapped
- womens-health-agent - Port 8002 mapped
- redis - Voice-specific Redis instance with TLS

Chat agent:

- agent - Port 8003, connects to FastAPI webhooks via internal Docker network

2.4 Data Stores

Component	Technology	Notes
Primary database	PostgreSQL 15 (Docker)	Database name: workflow_admin
Cache / queues	Redis 7.4 (Docker) x2	TLS enabled, no eviction policy
File storage	Azure Blob Storage	pdfs container
Secrets	Azure Key Vault	Existing vault

2.5 External Integrations

The following external services remain logically unchanged during migration:

- Twilio - Voice calls, webhooks, WebSocket media streams
- ElevenLabs - Voice agent text-to-speech integration
- Azure OpenAI - Large language models for chat and voice pipelines
- Azure Communication Services (ACS) - Transactional email
- Azure Storage - PDF and artifact persistence

3. Target Architecture (To-Be)

3.1 Design Principles

The target architecture adheres to the following core principles:

1. Isolation: All new infrastructure resides in resource group HRC-dev with Terraform state in HRC-dev-tfstate
2. No Production Coupling: Terraform does not reference or modify production resource group HRC
3. Greenfield Secrets: New dev Key Vault with secrets populated through controlled migration
4. Managed Services: Replace containerized PostgreSQL/Redis with Azure managed services
5. Infrastructure as Code: Single source of truth in HRC-infra repository
6. Minimal Application Changes: Identical Docker images and environment variable contracts

3.2 Resource Groups

Resource Group	Purpose
HRC-dev	All workload resources (ACA, ACR, PostgreSQL, Redis, Key Vault, Storage, monitoring)
HRC-dev-tfstate	Terraform remote state only (storage account hrcdevtfstate, container tfstate)
HRC (existing)	Production VM stack - unchanged in this phase

Region: westus2 (aligned with existing production region)

3.3 Target Azure Services

The following Azure services will be created in HRC-dev via Terraform. For more information, see [Azure Container Apps documentation](#).

Service	Role
Azure Container Apps Environment	Shared runtime environment for all application containers
Azure Container Registry (ACR)	Private container image registry for all services
Azure Container Apps (7 instances)	Frontend, Backend, Chat Agent, Voice Agents (2), PDF Worker, Scheduler
Azure Database for PostgreSQL Flexible Server	workflow_admin database with managed backups and scaling

Azure Cache for Redis	TLS-enabled cache and queue backend for workers and voice agents
Azure Key Vault (new, dev)	All dev environment secrets with RBAC and managed identity access
Storage Account (new, dev)	pdfs container for document storage
Log Analytics Workspace	Centralized log aggregation
Application Insights	Application performance monitoring and distributed tracing

4. Migration Approach

The migration will follow these phases:

1. Platform Provisioning: Create Azure infrastructure using Terraform (Container Apps, PostgreSQL, Redis, Key Vault, monitoring)
2. Secrets Configuration: Populate dev Key Vault with application secrets and connection strings
3. Application Deployment: Build Docker images, push to Azure Container Registry, update Container Apps
4. Parallel Testing: Run dev environment alongside production VM, validate all workflows
5. Data Migration: Move PostgreSQL data and Redis cache from VM to Azure managed services
6. Production Cutover: Update DNS and webhooks to point to new environment, maintain VM as backup

Key Risks

Main risks and how we will handle them:

- **Voice call interruptions:** Keep minimum warm containers running to avoid cold starts
- **Secret configuration errors:** Test all secrets before deployment, validate connections
- **Data migration issues:** Practice migration in test environment first, verify all data transfers
- **Service downtime:** Run new environment in parallel with current VM, test thoroughly before switching

5. Appendix

A. Key Vault Secret Names

All secrets follow kebab-case naming convention:

Application secrets:

- session-secret - Next.js session encryption key
- csrf-secret - CSRF token signing key
- websocket-auth-secret - WebSocket authentication token

Database and cache:

- postgres-connection-string - Full PostgreSQL connection string
- postgres-password - PostgreSQL admin password
- redis-url - Full Redis connection string
- redis-password - Redis authentication password

Azure services:

- azure-storage-connection-string - Storage account connection string
- azure-openai-endpoint - Azure OpenAI service endpoint URL
- azure-openai-api-key - Azure OpenAI API key
- acs-connection-string - Azure Communication Services connection string

B. Common Terms

Azure Container Apps (ACA): Managed service that runs our application containers in the cloud

Azure Container Registry (ACR): Private storage for our Docker container images

PostgreSQL: Database system that stores all application data

Redis: Fast cache system for temporary data and background jobs

Key Vault: Secure storage for passwords, API keys, and connection strings

Terraform: Tool that creates and manages Azure infrastructure using code

Docker Container: Packaged application that includes all its dependencies

WebSocket: Technology that keeps a live connection open for real-time voice calls

6. Summary

This document outlines the migration of the HRC healthcare outbound platform from a single virtual machine to modern Azure managed services. The new architecture uses Azure Container Apps to run our applications, managed PostgreSQL and Redis for data storage, and comprehensive monitoring.

Why This Change

The current system runs everything on one VM, which limits our ability to scale and makes maintenance more complex. Moving to Azure managed services gives us better reliability, easier scaling, and reduces operational overhead. The production system remains untouched during this migration.

What Changes

- Docker containers move from VM to Azure Container Apps
- PostgreSQL database moves to Azure managed PostgreSQL service
- Redis cache moves to Azure managed Redis service
- All application code stays the same - only infrastructure changes
- New monitoring tools (Log Analytics, Application Insights) provide better visibility

Next Steps

1. Set up Azure infrastructure using Terraform
2. Deploy applications to new environment
3. Test all voice call and chat workflows
4. Migrate data from current VM to Azure services
5. Switch production traffic to new environment

- End of document -